

Assignment 1

3-Bit Counter Driven by a 1 Hz Clock Signal

(Generated from an 8 MHz PLL Output via a Clock Divider)

Prepared by	Ahmed Yasser
Target Device	Zynq-7000 XC7Z020CLG484-2 (ZedBoard)
EDA Tool	Xilinx Vivado Design Suite
Document Type	Assignment Documentation

1. Objective

The goal of this assignment is to implement a 3-bit counter driven by a 1 Hz clock signal. The clock originates from the FPGA's internal oscillator, is fed into a Phase-Locked Loop (PLL) that generates an 8 MHz output clock, and is then divided down to 1 Hz using a clock divider module. Running the counter at such a slow clock rate allows its state transitions to be observed visually on the on-board FPGA LEDs.

2. Design Requirements

- Input clock sourced from the ZedBoard's internal 100 MHz oscillator.
- A Clocking Wizard (PLL) IP core used to generate an 8 MHz clock output.
- A custom clock-divider module that scales the 8 MHz clock down to 1 Hz.
- A 3-bit synchronous up-counter clocked by the divided 1 Hz signal.
- Counter output **counter[2:0]** mapped to the on-board user LEDs (LD0-LD2).

3. Target Hardware

The design was implemented and verified on a **ZedBoard** development board, which is built around the Xilinx/AMD **XC7Z020CLG484-2** Zynq-7000 SoC device. The corresponding board-level I/O physical constraints file (Master XDC) provided by Avnet/Digilent for the ZedBoard was used and adapted for this design.

4. Design Architecture

The RTL is organized into three cascaded blocks: the Clocking Wizard IP (**clk_wiz_0**) generating the 8 MHz clock from the input oscillator, a custom clock-divider module (**clock_divider**) producing the 1 Hz enable/clock, and the 3-bit counter module (**counter**) whose output drives the LEDs. The active-low reset (**rst_n**) is distributed to the PLL reset and, through the PLL locked signal, gates the reset of the downstream clock divider and counter.

4.1 RTL / Schematic Block Diagram

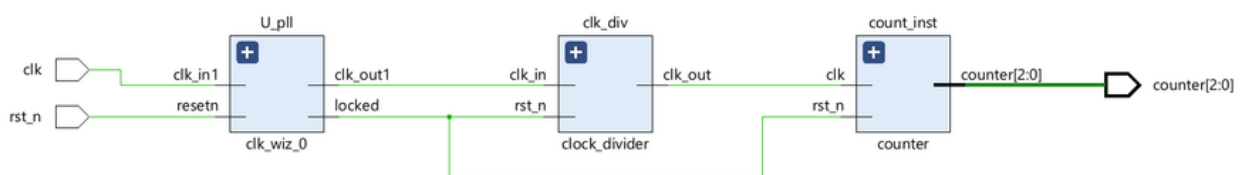


Figure 1: Elaborated RTL schematic showing the PLL, clock divider and counter cascade.

5. RTL Source Code

This section lists the Verilog source code for the three custom modules used in the design: the 3-bit counter, the clock divider, and the top-level module that instantiates the Clocking Wizard (PLL) IP together with the clock divider and the counter.

5.1 counter.v

Simple 3-bit synchronous up-counter with an asynchronous active-low reset.

```
module counter(clk, rst_n, counter);
    input clk;
    input rst_n;
    output reg [2 : 0] counter;

    always @ (posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            counter <= 3'b000;
        end
        else begin
            counter <= counter + 1'b1;
        end
    end
endmodule
```

5.2 clock_divider.v

Divides the 8 MHz PLL output clock down to 1 Hz by toggling clk_out every 4,000,000 input clock cycles (a 22-bit counter counting from 0 to 3,999,999), giving a full toggle period of 8,000,000 cycles at 8 MHz, i.e. 1 Hz.

```
module clock_divider(clk_in, rst_n, clk_out);
    input clk_in;
    input rst_n;
    output reg clk_out;

    reg [21 : 0] counter;

    always @ (posedge clk_in or negedge rst_n) begin
        if (!rst_n) begin
            counter <= 0;
            clk_out <= 0;
        end
        else begin
            if (counter == 3999999) begin
                counter <= 0;
                clk_out <= ~ clk_out;
            end
            else begin
                counter <= counter + 1'b1;
            end
        end
    end
endmodule
```

5.3 top.v

Top-level module instantiating the Clocking Wizard IP (clk_wiz_0), the clock divider, and the counter, wiring pll_clk, pll_rst and div_clk between them.

```
module top(clk, rst_n, counter);
    input clk;
    input rst_n;
    output [2 : 0] counter;

    wire pll_clk;
    wire pll_rst;
    wire div_clk;

    clk_wiz_0 U_pll
    (
        // Clock out ports
        .clk_out1(pll_clk),      // output clk_out1
        // Status and control signals
        .resetn(rst_n), // input resetn
        .locked(pll_rst),      // output locked
        // Clock in ports
        .clk_in1(clk)          // input clk_in1
    );

    clock_divider clk_div (.clk_in(pll_clk),
                          .rst_n(pll_rst),
                          .clk_out(div_clk)
    );

    counter count_inst (.clk(div_clk),
                      .rst_n(pll_rst),
                      .counter(counter)
    );
endmodule
```

5.4 top_tb.v

```
module top_tb();
    reg clk;
    reg rst_n;
    wire [2 : 0] counter;

    top dut(
        .clk(clk),
        .rst_n(rst_n),
        .counter(counter)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~ clk;
    end

    initial begin
        rst_n = 1'b0;
        #50;
        rst_n = 1'b1;

        #2000000;
        $finish;
    end

    initial begin
        $display("Time\tReset\tCounter");
        $monitor("%0t\t%b\t%h", $time, rst_n, counter);
    end
endmodule
```

6. I/O Physical Constraints (XDC)

The design targets the ZedBoard, and its corresponding Master XDC constraints file was used to bind the design ports to the correct physical package pins and I/O standards. The relevant pin assignments used by this design are summarized below.

Signal	Board Resource	Package Pin	I/O Standard
clk	GCLK (100 MHz osc.)	Y9	LVC MOS33 (Bank 13)
rst_n	SW0 (DIP switch)	F22	LVC MOS18 (Bank 35)
counter[0]	LD0 (User LED)	T22	LVC MOS33 (Bank 33)
counter[1]	LD1 (User LED)	T21	LVC MOS33 (Bank 33)
counter[2]	LD2 (User LED)	U22	LVC MOS33 (Bank 33)

6.1 Timing Constraints

A primary 100 MHz clock constraint was applied to the input clk port, and a generated clock constraint was derived on the PLL output (U_pll/clk_out1) at the 8 MHz rate. False-path constraints were applied from the asynchronous reset (rst_n) to the counter's clear inputs, and from the counter register clock pins to the counter output ports, since these paths are not timing-critical single-bit toggle observations.

```
create_clock -name clk -period 10 [get_ports clk]

create_generated_clock -name pll_clk \
    -source clk \
    -divide_by [expr 100/8] \
    U_pll/clk_out1

set_false_path -from rst_n -to counter_reg[0]/CLR
set_false_path -from rst_n -to counter_reg[1]/CLR
set_false_path -from rst_n -to counter_reg[2]/CLR

set_false_path -from counter_reg[0]/C -to counter[0]
set_false_path -from counter_reg[1]/C -to counter[1]
set_false_path -from counter_reg[2]/C -to counter[2]
```

6.2 I/O Standards by Bank

- Bank 13 (clk): LVC MOS33 - fixed 3.3V supply on ZedBoard.
- Bank 33 (counter[2:0] / LEDs): LVC MOS33 - fixed 3.3V supply on ZedBoard.
- Bank 35 (rst_n / DIP switch): LVC MOS18 - 1.8V Vadj default setting.

7. Behavioral Simulation

A behavioral simulation was run to verify functional correctness before synthesis. The waveform below shows the input clock (clk), the active-low reset chain (rst_n, pll_rst), the 8 MHz PLL output (pll_clk), the divided clock (div_clk), and the resulting 3-bit counter value.

Note on simulation clock rate: the final design divides the 8 MHz PLL clock down to 1 Hz for real-time observation of the LEDs on the actual board. However, simulating a full 1 Hz period in a behavioral testbench would require simulating on the order of seconds of real time, which translates into a very large number of simulation cycles and an impractically long run time / waveform database. For this reason, the clock-divider parameters were temporarily scaled up for simulation purposes only, so that div_clk toggles at **1 MHz** instead of 1 Hz. This keeps the functional behavior identical (the counter still increments once per div_clk rising edge and wraps modulo 8) while keeping the simulation time manageable. The actual hardware implementation still uses the 1 Hz divide ratio described in Section 5.2.

7.1 Waveform (Simulated at 1 MHz)

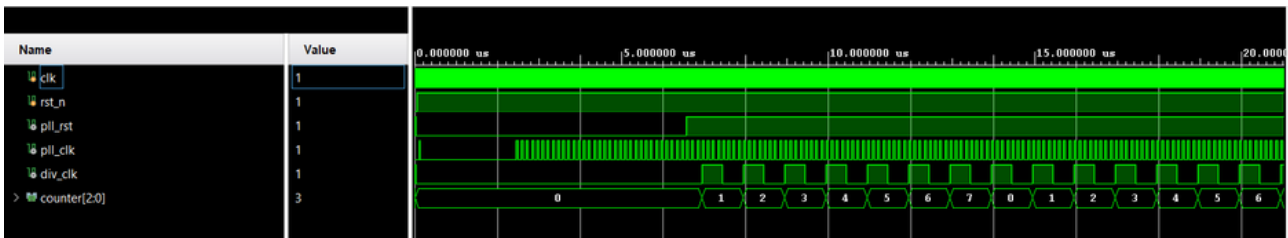


Figure 2: Simulation waveform (div_clk at 1 MHz for simulation speed) - the counter increments on every rising edge of the divided clock, wrapping from 7 back to 0.

As shown in the waveform, once the PLL locks and the reset is released, the 8 MHz pll_clk toggles continuously, and div_clk produces one pulse per divider period (1 MHz in this simulation run). The counter[2:0] value increments synchronously with div_clk and correctly wraps around from 7 (111) back to 0 (000), confirming proper 3-bit modulo-8 counting behavior. The same logic behavior is expected at the real 1 Hz operating rate on hardware.

8. Synthesis Results

After synthesis, Vivado generates the gate-level schematic connecting the PLL (clk_wiz_0), the clock

divider, and the counter, with output buffers (OBUF) driving the three counter output pins connected to the LEDs, and an input buffer (IBUF) on the reset input.

8.1 Synthesized Schematic

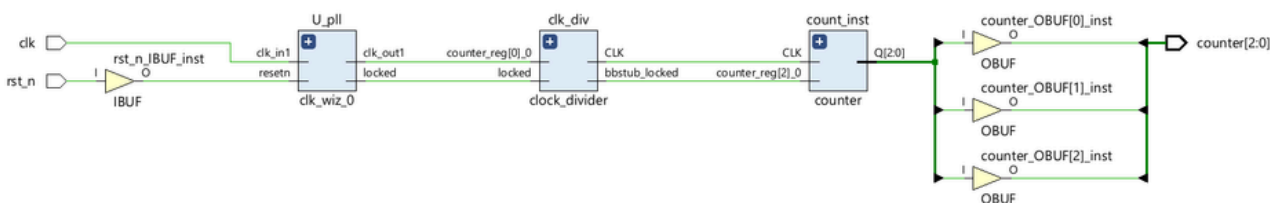


Figure 3: Post-synthesis schematic showing IBUF/OBUF insertion at the design boundary.

9. Implementation Results

9.1 Timing Summary

The design timing report confirms that all user-specified timing constraints are met, with zero failing endpoints across setup, hold, and pulse-width checks.

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 116.544 ns		Worst Hold Slack (WHS): 0.115 ns		Worst Pulse Width Slack (WPWS): 2.633 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 23		Total Number of Endpoints: 23		Total Number of Endpoints: 29	
All user specified timing constraints are met.					

Figure 4: Design timing summary - WNS, WHS and WPWS are all positive with zero failing endpoints.

9.2 Resource Utilization

Given the simplicity of the design (a clock divider and a 3-bit counter), resource utilization is extremely low relative to the XC7Z020's available fabric, using only 31 Slice LUTs and 26 Slice Registers.

Name	^1	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	Bonded IOB (200)	BUFGCTRL (32)	PLLE2_ADV (4)
▼ N top		31	26	16	31	5	2	1
clk_div (clock_divider)		29	23	15	29	0	0	0
count_inst (counter)		2	3	1	2	0	0	0
> U_pll (clk_wiz_0)		0	0	0	0	0	2	1

Figure 5: Post-implementation utilization breakdown by hierarchical module.

9.3 Device Layout

The implemented device view below shows the placement of the small logic footprint within the XC7Z020 fabric, along with the clocking resources (BUFGCTRL) and the PLL primitive (PLLE2_ADV) used by the Clocking Wizard IP.

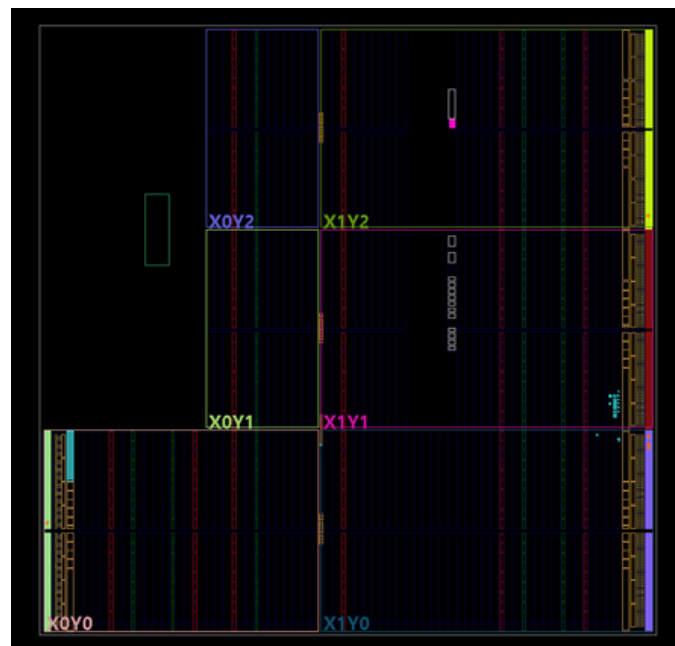


Figure 6: Implemented device layout on the XC7Z020CLG484-2.